

## Objectif

Développer une application web permettant à un utilisateur de réserver des sessions de tests de langues. L'application doit permettre à l'utilisateur de s'authentifier, de consulter et gérer ses réservations, et d'accéder à la liste des sessions disponibles. Le backend sera réalisé avec Symfony pour exposer une API REST, et le frontend utilisera Next.js et React. Spécifications techniques

### 1. Frontend : Next.js / React

Utiliser Next.js et React pour créer une interface utilisateur responsive.

Fonctionnalités requises :

Page de connexion : Authentification via JWT. Après la connexion, l'utilisateur est redirigé vers la page de gestion de ses réservations.

Page de gestion du compte utilisateur :

Affichage des informations personnelles (nom, email).

Modification des informations du compte (nom, email).

Page de réservation des sessions :

Affichage d'une liste des sessions de tests disponibles (langue, date, heure, lieu).

Réservation d'une session disponible.

Affichage de la liste des réservations de l'utilisateur.

Annulation d'une réservation.

Utilisation d'Axios ou Fetch API pour tous les appels à l'API REST Symfony.

Pagination sur la liste des sessions.

### 2. Backend : Symfony

Créer une API REST avec Symfony pour gérer les utilisateurs, les sessions, et les réservations.

Fonctionnalités à implémenter :

Authentification des utilisateurs :

Enregistrement et connexion avec JWT.

Middleware de protection des routes pour les utilisateurs authentifiés.

Gestion des utilisateurs :

CRUD minimal : enregistrement, mise à jour des informations (nom, email).

Gestion des sessions de tests :

CRUD des sessions : créer, lire, modifier et supprimer des sessions de test (langue, date, heure, lieu, nombre de places).

Pagination sur la liste des sessions.

Gestion des réservations :

Un utilisateur peut réserver une session (si places disponibles).

Un utilisateur peut consulter et annuler ses propres réservations.

Un utilisateur ne peut réserver qu'une fois la même session.

Validation des données reçues dans les requêtes avec les outils de validation de Symfony.

### 3. Base de données : MongoDB

Utiliser MongoDB pour stocker les données :

Collection Utilisateurs : (id, nom, email, mot de passe hashé)

Collection Sessions : (id, langue, date, heure, lieu, nombre de places)

Collection Réservations : (id, session\_id, user\_id, date de réservation)

### Livrables

Code source complet :

Code structuré et bien documenté.

Projet comprenant un répertoire pour le frontend (Next.js) et un répertoire pour le backend (Symfony).

Fichiers .env ou équivalent pour les configurations (connexion à MongoDB, secret JWT, etc.).

Tests unitaires et/ou d'intégration :

Avec PHPUnit pour le backend Symfony.

Avec Jest (ou équivalent) pour le frontend.

Dockerisation :

Fournir un fichier docker-compose.yml pour faciliter le déploiement et l'exécution du projet.

Documentation d'installation et d'exécution :

Instructions claires pour installer et exécuter l'application dans un fichier README.md.

Critères d'évaluation

Fonctionnalité : Toutes les fonctionnalités demandées sont présentes et fonctionnelles.

Qualité du code : Code lisible, structuré, conforme aux bonnes pratiques.

Sécurité : Authentification JWT et gestion correcte des droits d'accès.

Performance : Application performante et réactive.

Documentation : Projet bien documenté avec des instructions claires.

Tests : Couverture de tests pour le frontend et le backend.

Docker : Projet exécutable facilement via Docker.

Instructions pour le candidat

Hébergez le projet sur une plateforme comme GitHub et fournissez un lien vers le dépôt.

Assurez-vous que le projet soit exécutable facilement, avec des instructions claires dans un fichier README.md.